



Funzionamento di un programma

Massimo Esposito



CPU ed esecuzione dei programmi

Sommario

1. CPU ed esecuzione dei programmi
2. Dal linguaggio macchina ai linguaggi ad alto livello
3. Sintassi e traduzione dei programmi

Il ruolo della CPU

La **CPU** è spesso definita il cervello del computer, ma è in realtà un **dispositivo elettronico** che esegue semplici operazioni.

Queste comprendono la lettura di dati dalla memoria centrale, operazioni aritmetiche e logiche, spostamento di dati da una locazione di memoria a un'altra.

Tutte queste operazioni sono eseguite seguendo le **istruzioni** dei programmi.

Le istruzioni

Ogni **istruzione** di un programma è un comando che indica alla CPU di svolgere una determinata operazione, come sommare due numeri o confrontare valori.

Una istruzione è codificata nella forma di **sequenza di 0 e 1**.

La CPU può comprendere solamente istruzioni scritte in **linguaggio macchina** e le istruzioni in linguaggio macchina hanno tutte una struttura binaria.

Il set di istruzioni della CPU

Ogni **operazione** che una CPU può eseguire corrisponde a una specifica istruzione in linguaggio macchina.

L'insieme completo di queste istruzioni costituisce il **set di istruzioni** della CPU, ed è unico per ciascun tipo di processore.

Questo set determina le capacità operative della CPU e rappresenta **l'interfaccia** tra l'hardware e i programmi che lo controllano.

L'importanza della sequenza di istruzioni

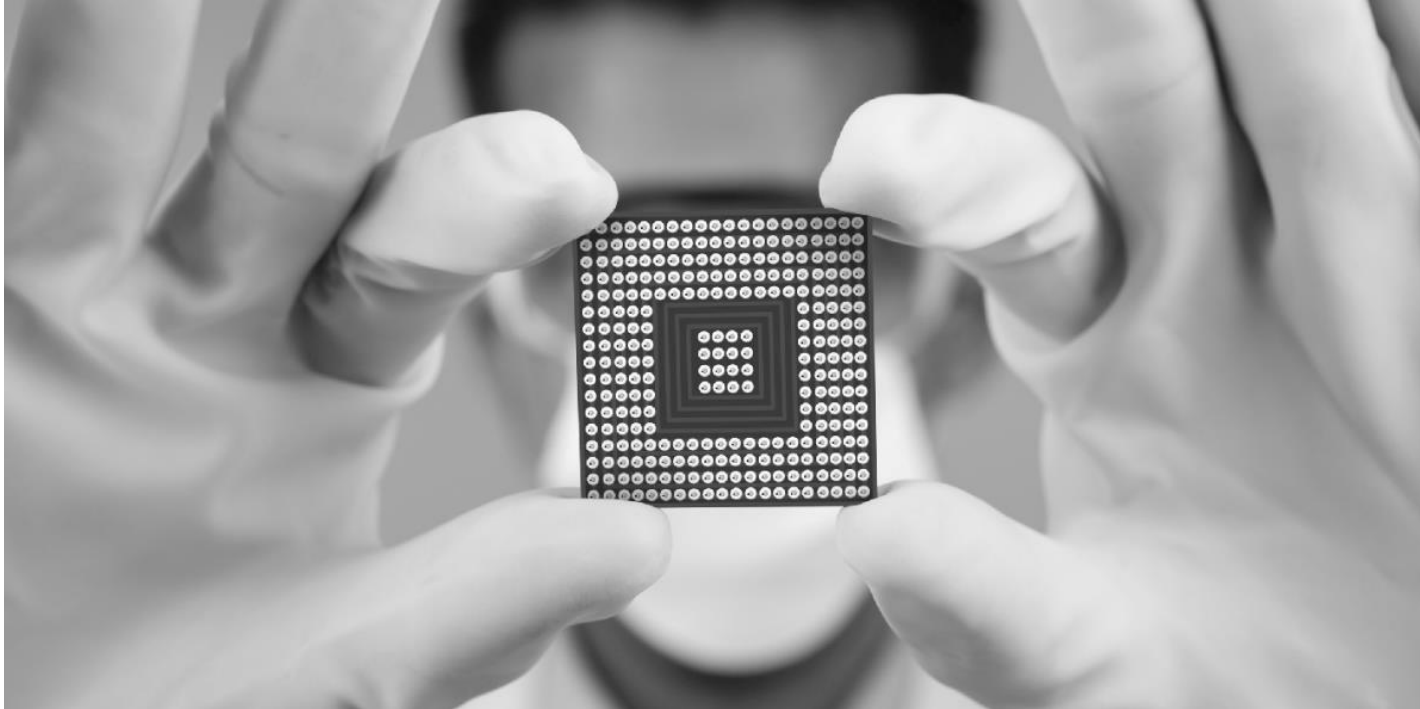
Singole istruzioni eseguono operazioni semplici, ma per ottenere risultati significativi è necessaria una **sequenza ordinata di istruzioni**.

- Attività complesse, come il calcolo di un interesse annuo, richiedono l'esecuzione di migliaia o milioni di istruzioni.

I programmi sono **strutture complesse** fatte di istruzioni elementari, eseguite in modo rapido e preciso dalla CPU.

- Il valore di un programma dipende dalla corretta combinazione e sequenza delle istruzioni, che la CPU deve seguire rigorosamente.

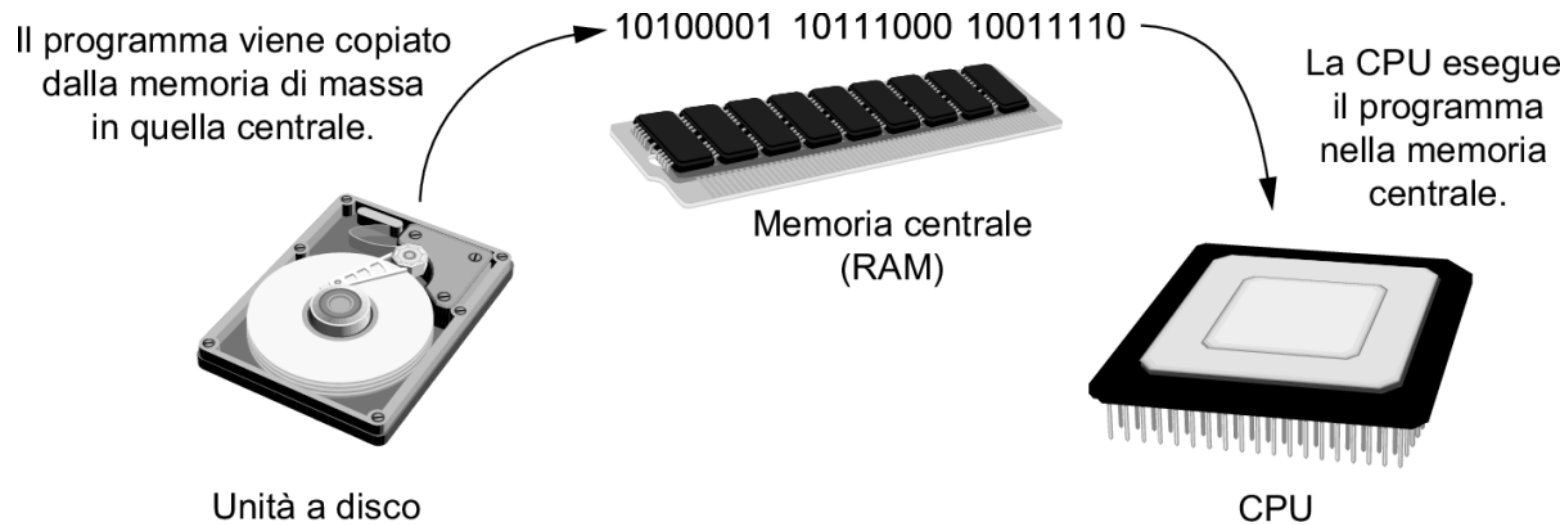
Linguaggio macchina e microprocessori



Oggi esistono numerose aziende produttrici di microprocessori, tra cui le più famose sono **Intel, AMD e Motorola**.

- Ogni produttore progetta microprocessori con un set di istruzioni specifico, eseguibile solo dai propri dispositivi.
- I programmi devono essere scritti o adattati per il set di istruzioni del microprocessore di destinazione.

Esecuzione dei programmi

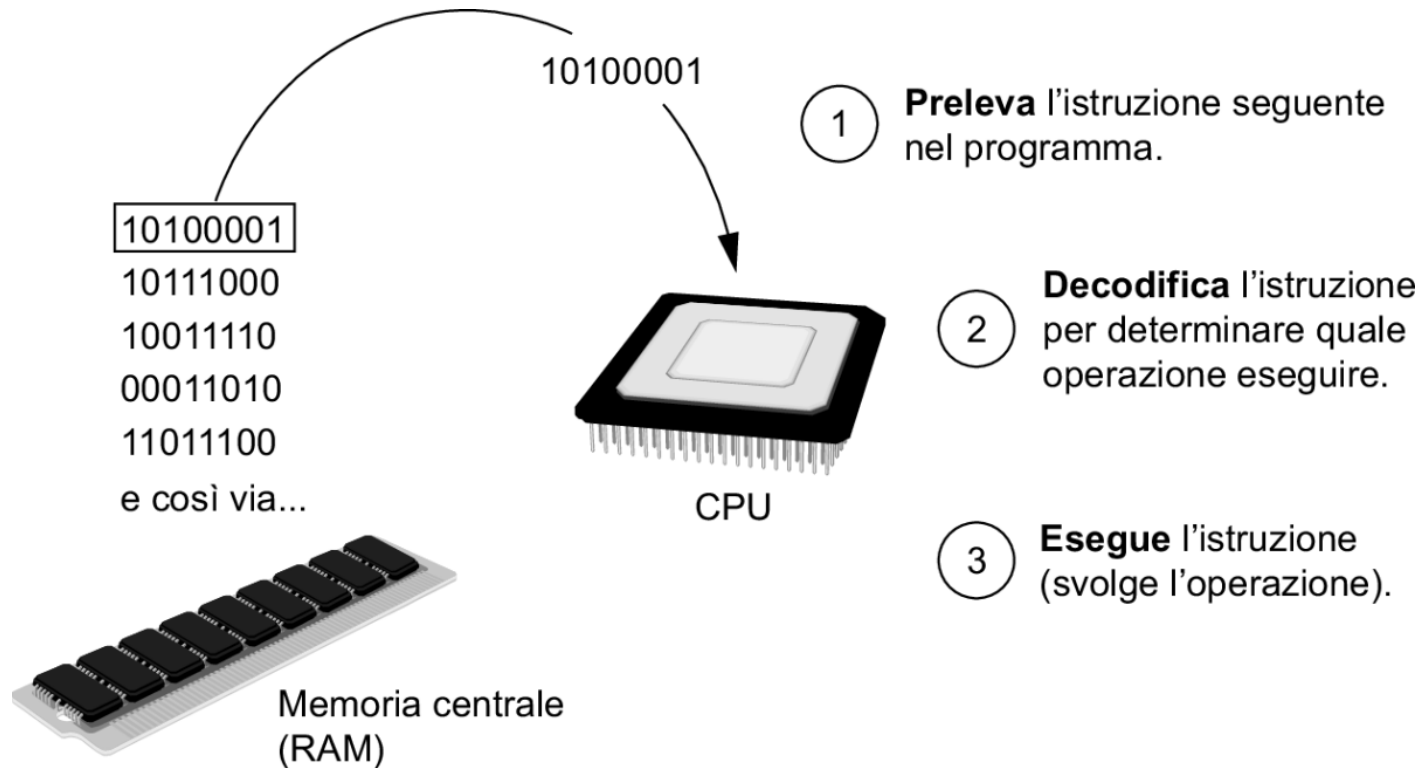


1. I programmi vengono **memorizzati** in dispositivi di **memoria di massa** (es. dischi rigidi) da cui possono essere scaricati via internet o app store.
2. Sono successivamente copiati dalla memoria di massa alla **RAM**, ossia la **memoria centrale** del computer per poter essere eseguiti.
3. Quando l'utente **avvia** un programma, ad esempio facendo doppio clic sull'icona, il sistema operativo provvede a caricarlo nella RAM.

Il ciclo fetch-decode-execute

- Il ciclo di esecuzione di un programma prende il nome di **ciclo fetch-decode-execute** (preleva, decodifica, esegui)
- Si divide in tre fasi: **fetch** (prelievo dell'istruzione dalla memoria), **decode** (decodifica dell'istruzione) ed **execute** (esecuzione).
-
- Questo processo si ripete per ogni istruzione del programma.

Il ciclo fetch-decode-execute in dettaglio



1. **Fetch:** la CPU preleva dalla memoria l'istruzione successiva in linguaggio macchina da eseguire.
2. **Decode:** la CPU interpreta l'istruzione binaria appena prelevata dalla memoria per determinare quale operazione deve eseguire.
3. **Execute:** la CPU esegue l'operazione indicata, completando il ciclo.



Dal linguaggio macchina ai linguaggi ad alto
livello

Limiti del linguaggio macchina

I computer **eseguono** solo programmi scritti in **linguaggio macchina**, ovvero lunghissime sequenze di istruzioni binarie.

Scrivere in linguaggio macchina è complicato, soggetto a errori e poco leggibile.

- Anche una minima variazione, come scambiare uno 0 con un 1, può causare **malfunzionamenti** nel programma.

Introduzione al linguaggio assembly

- Per facilitare la scrittura dei programmi è stato introdotto il **linguaggio assembly**, un'alternativa più leggibile al codice binario
- Utilizza **mnemonici** come ``add``, ``mov``, ``mul`` invece delle sequenze binarie.
- Questo approccio è più comprensibile per i programmatori.

Il ruolo dell'assembler

Programma in linguaggio
Assembly

```
mov eax, Z  
add eax, 2  
mov Y, eax  
  
e così via...
```



Assembler



Programma in linguaggio
macchina

```
10100001  
10111000  
10011110  
  
e così via...
```

I programmi scritti in linguaggio assembly devono essere tradotti in linguaggio macchina da un **assembler**.

Questo programma converte il codice assembly in una versione binaria eseguibile dalla CPU.



Limiti del linguaggio assembly

Il linguaggio assembly, pur facilitando la scrittura rispetto al binario, rimane **complesso e difficile da gestire**.

- È un sostituto diretto del linguaggio macchina e richiede una profonda conoscenza della CPU per essere utilizzato efficacemente.
- Anche i programmi più semplici richiedono numerose istruzioni, rendendo lo sviluppo lungo e soggetto a errori.

Per queste caratteristiche, l'assembly viene definito un **linguaggio a basso livello**, vicino all'hardware ma poco pratico.

Evoluzione verso l'alto livello

I **linguaggi ad alto livello** sono nati negli anni '50 per permettere ai programmatori di concentrarsi sulla **logica** e non sulla struttura della CPU.

L'uso di istruzioni sintetiche e leggibili, più simili al **linguaggio umano**, ha reso la programmazione più accessibile ed efficiente.

- Esempio: in COBOL, per visualizzare un messaggio sullo schermo, basta scrivere **DISPLAY "Hello world"**.

Vantaggi dei linguaggi ad alto livello

I linguaggi ad alto livello, come **Python**, permettono la scrittura di codice **chiaro e conciso**.

Un semplice comando può svolgere operazioni complesse che in assembly richiederebbero molte righe.

- In Python, il comando `print('Hello world')` stampa un messaggio a schermo.
- In linguaggio assembly servirebbero decine di righe.

Questo evidenzia la **potenza** e l'**efficienza** dei linguaggi ad alto livello.



Sintassi e traduzione dei programmi

Parole chiave nei linguaggi

Ogni linguaggio ha un insieme di **parole predefinite** che hanno un significato preciso e non possono essere usate in altro modo.

Le parole che compongono un linguaggio di programmazione ad alto livello sono chiamate **parole chiave** o **parole riservate**.

- In Python, parole come ``if``, ``else``, ``print`` sono esempi di parole chiave riservate.

Operatori

- Oltre alle parole chiave, i linguaggi di programmazione utilizzano **operatori** per eseguire operazioni sui dati.
- Gli **operatori aritmetici** sono presenti in tutti i linguaggi: in Python, ad esempio, + serve per sommare due numeri.
- L'istruzione $12 + 75$ produce il risultato della somma tra i due valori.
- Python include molti altri operatori, che permettono di **manipolare** dati in modo versatile e potente.

Sintassi

Ogni linguaggio ha una propria **sintassi**, cioè un insieme di regole formali per scrivere correttamente le istruzioni.

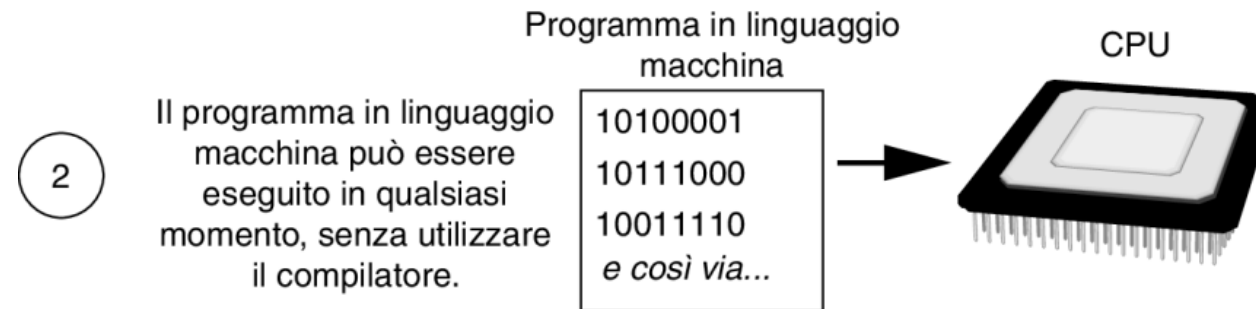
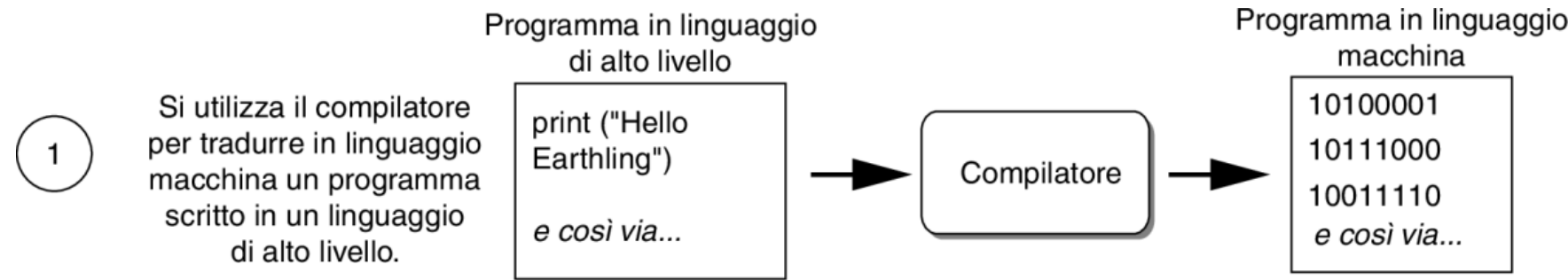
- La sintassi stabilisce il corretto ordine di parole chiave, operatori e segni di punteggiatura all'interno di un'istruzione.
- Le istruzioni (o statements) sono le unità fondamentali di un programma: eseguono operazioni complete e coerenti.

Apprendere un linguaggio di programmazione significa impararne le regole sintattiche per scrivere **codice corretto ed efficace**.

Compilatori e interpreti

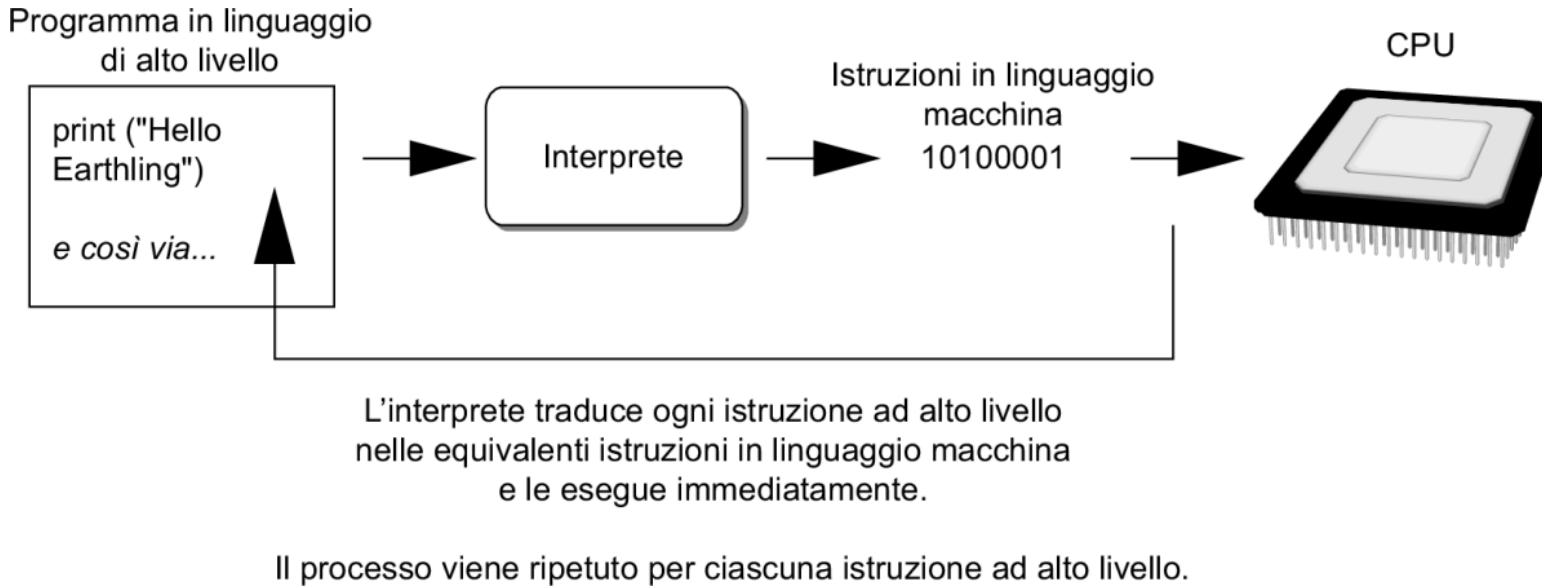
- La CPU può eseguire solo istruzioni in **linguaggio macchina**, quindi i programmi scritti in linguaggi ad alto livello devono essere prima **tradotti**.
- Questa **traduzione** è necessaria affinché istruzioni comprensibili all'essere umano diventino **eseguibili** per l'hardware.
- La traduzione può avvenire tramite **compilatori** o **interpreti**, strumenti essenziali per l'esecuzione del software moderno.

Il compilatore



1. Un **compilatore** traduce l'intero programma da un linguaggio ad alto livello a linguaggio macchina, producendo un file eseguibile.
2. Una volta compilato, il programma può essere **eseguito ogni volta** senza bisogno di ulteriori traduzioni.
3. I processi di compilazione ed esecuzione sono **distinti e separati**: prima si traduce, poi si esegue.

L'interprete



1. L'interprete **traduce ed esegue** una istruzione alla volta: legge, converte in linguaggio macchina, e subito la esegue.
2. A differenza del compilatore, l'interprete **non crea file eseguibili** separati: l'esecuzione è diretta e continua.
3. Questo approccio consente un **ciclo rapido** di test e modifica del codice, ideale per l'apprendimento e lo sviluppo agile.

Codice sorgente ed errori di sintassi

Il codice scritto in un linguaggio ad alto livello si chiama **codice sorgente** o semplicemente **sorgente**.

- Questo codice viene scritto in un **editor di testo** e salvato su disco prima di essere interpretato o compilato.

Se il codice contiene un **errore di sintassi**, come una parola chiave errata o una parentesi mancante, la traduzione si interrompe.

- Compilatori e interpreti bloccano l'esecuzione e restituiscono un messaggio di errore, utile per comprendere cosa correggere.

Bibliografia

Libro di testo

- Tony G. (2022). Introduzione a Python - 5/Ed (Capitolo 1, pp. 36-43). Pearson Italia spa. (Disponibile nella sezione “Biblioteca”)

Crediti

- Tutte le immagini utilizzate in questa presentazione sono tratte dal libro di testo qui indicato in bibliografia.